

Automatic Documentation Methodology (C + Python)

with Sphinx, Doxygen, Breathe and Read the Docs

Project: SDR Sensor

Team: *(Fill names)*

February 10, 2026

Abstract

This document describes, in a conceptual way, the methodology used to unify the automatic documentation of a hybrid repository (C and Python code) into a single publishable site. The approach integrates: (i) extraction of documentation from Python docstrings, (ii) extraction of documentation from C comments and signatures, and (iii) final assembly in a Sphinx-generated website, compatible with Read the Docs deployment.

1 Context and problem

The project worked on two documentation lines developed in separate copies of the same repository:

- **Copy A (C-focused):** aimed at documenting `.c/.h` files using Doxygen and its structured output.
- **Copy B (Python-focused):** aimed at documenting `.py` modules using autodocumentation (docstrings) rendered by Sphinx.

The problem was to **merge both approaches** to obtain a **single documentation site** (same navigation tree, same visual theme, same configuration) where both the C API and the Python API coexist.

2 Goal

- Centralize documentation under a single `docs/` tree with one Sphinx configuration.
- Preserve automatic generation for both languages:
 - **Python:** from docstrings and module/class/function signatures.
 - **C:** from Doxygen (structured output) embedded into Sphinx through Breathe.
- Enable reproducible builds locally and on a documentation hosting service (e.g., Read the Docs).

3 Solution architecture

The unification is based on **clearly separating sources of truth** and **connecting both worlds through a stable integration point**:

- **Python source of truth:** docstrings (Google/NumPy style) + Sphinx code introspection.
- **C source of truth:** Doxygen-style comments/tags in `.h/.c`.
- **Integration:** Doxygen produces a *structured representation* and Breathe allows Sphinx to consume and display it within the final site.

3.1 Components and responsibilities

Component	Role in the pipeline
Sphinx	Site orchestrator: processes <code>.rst</code> , resolves references, and generates final HTML.
Autodoc / Napoleon	Extract and format documentation from Python docstrings (classes, functions, attributes).
Doxygen	Analyzes C/C++ and produces structured output from comments/signatures (ideally XML).
Breathe	Bridges Doxygen output into Sphinx; provides directives to “include” C entities in <code>.rst</code> pages.
Visual theme (e.g., Furo)	Defines website look and feel (navigation, typography, sidebar, etc.).
Optional extensions (e.g., Mermaid)	Add diagrams and enriched content if the project requires it.

4 Unified workflow (step-by-step, conceptual)

The logical workflow is organized in phases, prioritizing reproducibility and traceability:

4.1 Phase 1: Documentation structure normalization

- Define a **single documentation directory** (e.g., `docs/`).
- Under `docs/source/`, organize content into coherent sections:
 - System chapters (architecture, installation, operation).
 - Python API.
 - C API (e.g., under `c_api/` with its own `index.rst`).
- Define a **main toctree** that links both APIs and all remaining chapters.

4.2 Phase 2: Structured artifact generation for C

- Configure Doxygen to **traverse the real folders** containing the C code to document.
- Generate Doxygen output in a structured format suitable for integration (typically XML).
- Validate that the output includes an entry index (anchor point) and entity artifacts (structs, functions, defines, typedefs).

4.3 Phase 3: C integration into Sphinx

- Configure Sphinx to know the path to the Doxygen output.
- Enable Breathe to expose directives that **render C entities** inside `.rst` pages.
- Build `.rst` pages that act as “bridges” between site navigation and C objects.

4.4 Phase 4: Python automatic documentation

- Sphinx (with autodoc and napoleon) imports Python code and **extracts docstrings**.
- The final site publishes the Python API consistently (modules, classes, functions).
- Difficult external dependencies are handled via isolation techniques (so builds do not fail due to missing heavy libraries).

4.5 Phase 5: Final assembly, validation, and publication

- Sphinx generates the final HTML by integrating global navigation with both APIs.
- Review typical warnings: ambiguous references, duplicates, missing entities, wrong paths.
- Prepare deployment on a reproducible hosting environment (e.g., Read the Docs), ensuring dependencies are installed and the pipeline runs end-to-end.

5 Pipeline flowchart

6 Verification strategy (conceptual “checks”)

To ensure automation actually documents what it should, verification is based on observable evidence:

- **Input verification:** confirm target C/Python folders exist and contain expected files.
- **Artifact verification:** ensure the C structured output contains an entry index and enough objects (structs/functions) to be referenced.
- **Reference resolution verification:** detect ambiguities when the same name is published from different paths/domains.
- **Site consistency verification:** ensure navigation links to the new sections (C API) and the final render does not leave orphan pages.

7 Common risks and how they are handled (conceptual)

- **Unknown directives:** typically means the responsible extension is not enabled (e.g., the layer that integrates Doxygen with Sphinx).
- **C entities “not found”:** often due to incomplete structured output (wrong input paths or missing output generation).
- **Duplicate/ambiguous references:** occur when the same object is documented from multiple paths or indices see equivalent definitions.

- **Non-compatible docstring syntax:** some markup may not be supported; describing attributes/fields with supported structured text is safer than using non-standard directives.

8 Conclusions

The methodology successfully merged two complementary approaches:

- **Native Python documentation** based on docstrings and introspection.
- **Structured C documentation** based on static analysis with Doxygen.

The result is a single, extensible, publishable pipeline where documentation stays close to the code, reducing manual effort and improving long-term consistency.

References (methodology and tools)

References

- [1] Sphinx Documentation. *Automatic documentation generation (autodoc)*.
<https://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html>
- [2] Sphinx Documentation. *Napoleon (Google/NumPy docstrings)*.
<https://www.sphinx-doc.org/en/master/usage/extensions/napoleon.html>
- [3] Doxygen Manual. *Configuration options (incl. OUTPUT_DIRECTORY, GENERATE_XML, XML_OUTPUT)*.
<https://www.doxygen.nl/manual/config.html>
- [4] Breathe Documentation. *Breathe: reStructuredText directives for Doxygen output*.
<https://breathe.readthedocs.io/en/latest/directives.html>
- [5] Read the Docs Documentation. *Deploying Sphinx on Read the Docs*.
<https://docs.readthedocs.com/platform/stable/intro/sphinx.html>
- [6] PyPI. *sphinxcontrib-mermaid: Mermaid diagrams for Sphinx*.
<https://pypi.org/project/sphinxcontrib-mermaid/>
- [7] Furo Theme Documentation. *Furo: Sphinx documentation theme*.
<https://pradyunsg.me/furo/>

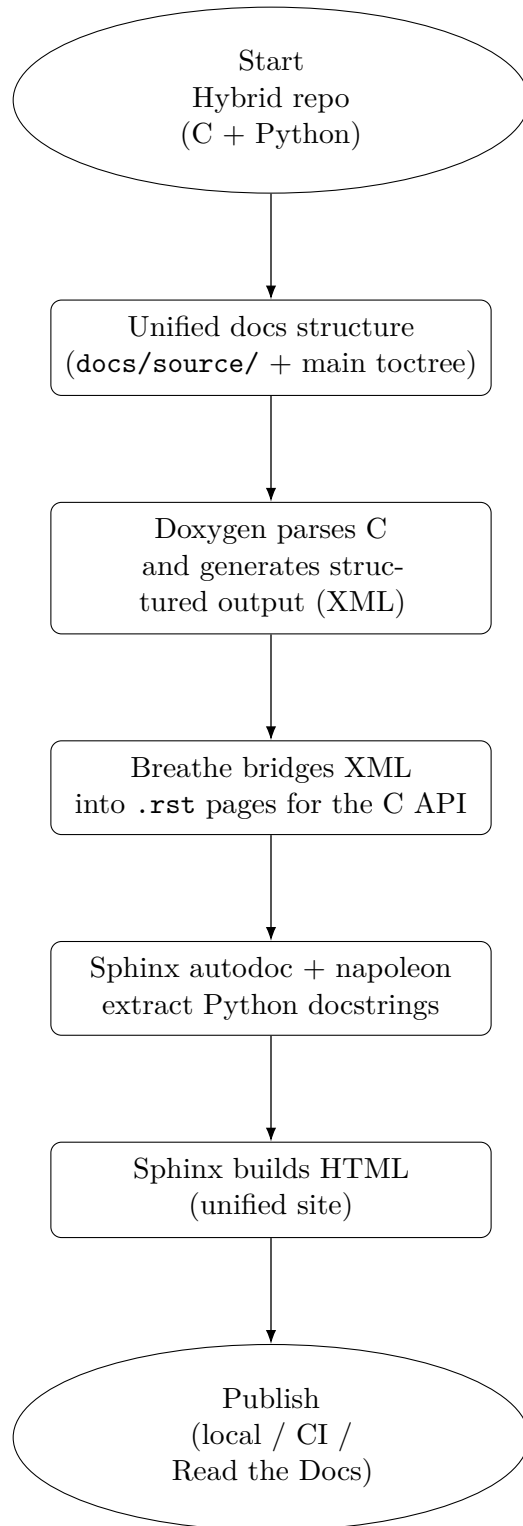


Figure 1: Conceptual pipeline for automatic documentation in a hybrid repository.